FPGA Capstone

Paper Presentation Summary

FPGA Dynamic and Partial Reconfiguration: A Survey of Architectures, Methods, and Applications

KIZHEPPATT VIPIN, Nazarbayev University, Kazakhstan

SUHAIB A. FAHMY, University of Warwick, United Kingdom

https://doi.org/10.1145/3193827

ACM Computing Surveys: Published: 25 July 2018

Arun Murugappan, Pranav T Varkey, Shrikrishna Pandit

Table of Contents

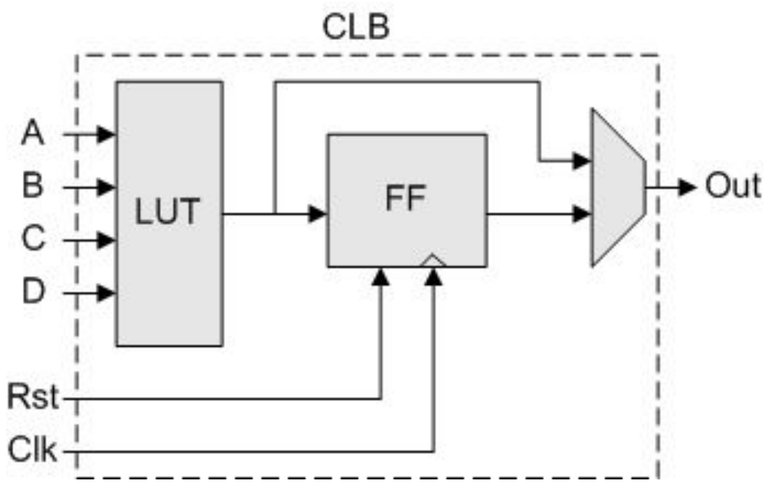
- Introduction
- FPGA & Reconfigurable Architecture
- Design, Implementation & Simulation tool support
- Technical Challenges & Tool chain gaps
- Overhead Reduction Methods
- Runtime Management
- Future
- Conclusion

Motivation

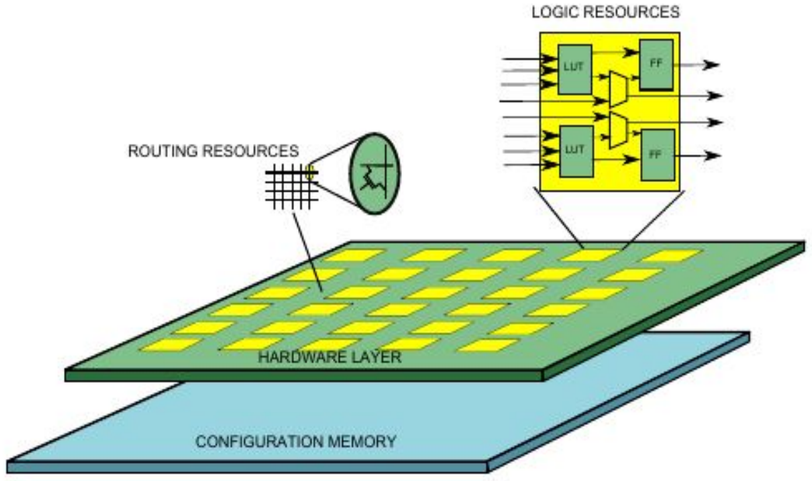
To highlight the gap between academic research and real-world deployment of reconfigurable system by investigating the existing architectures, design flows, applications and identifying the key challenges in making reconfigurable FPGA system easier to design

FPGA Architecture

- FPGA consists of Configurable Logic Block (CLB) surrounded by interconnects,DSPs , RAMs, I/O, ports etc
- CLBs contains Look Up Tables(LUTs) which stores predefined data output for any combination of inputs.
- The device can be split into Hardware layer & Configuration Memory layer.
- Hardware Layer: Compute elements - DSPs, CLBs, Multipliers, interconnects etc
- Configuration Memory Layer contains Configuration information of the Hardware Layer



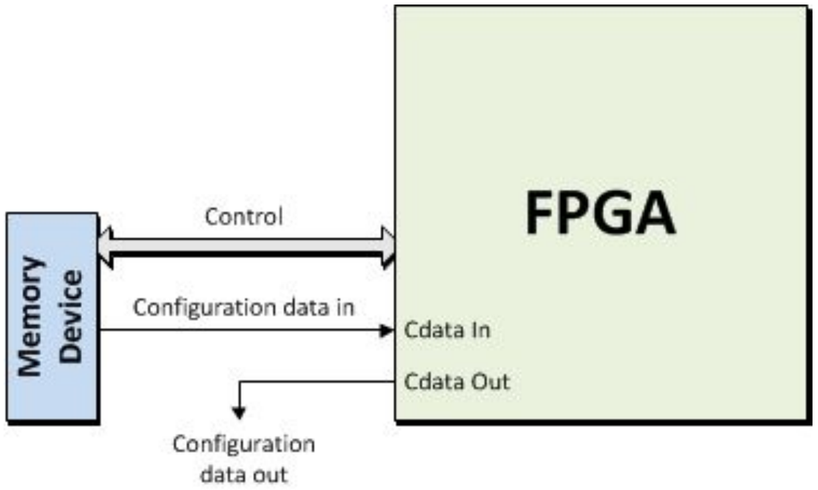
Simple CLB with 4 input LUT



Layers of a FPGA

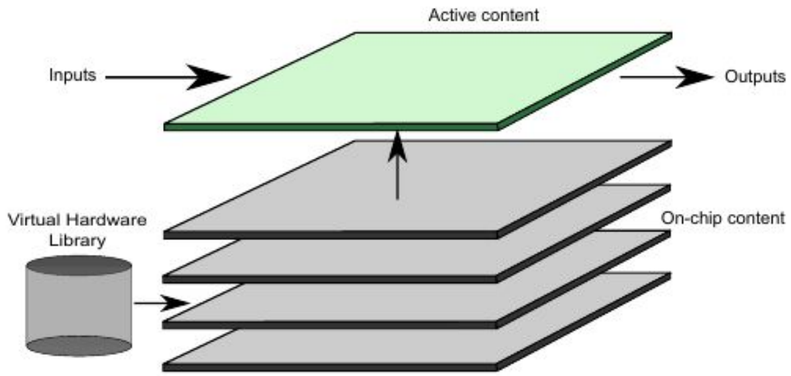
Configuration and Reconfiguration of FPGA

- Bitstream - Binary sequence of data containing codes for all of the configuration information of a given FPGA
- Configuration refers to the process of writing data onto the configuration Memory plane.
- Reconfiguration is replacement of a old bitstream with a new bitstream.
- Reconfiguration -
    - Full Reconfiguration
    - Partial Reconfiguration



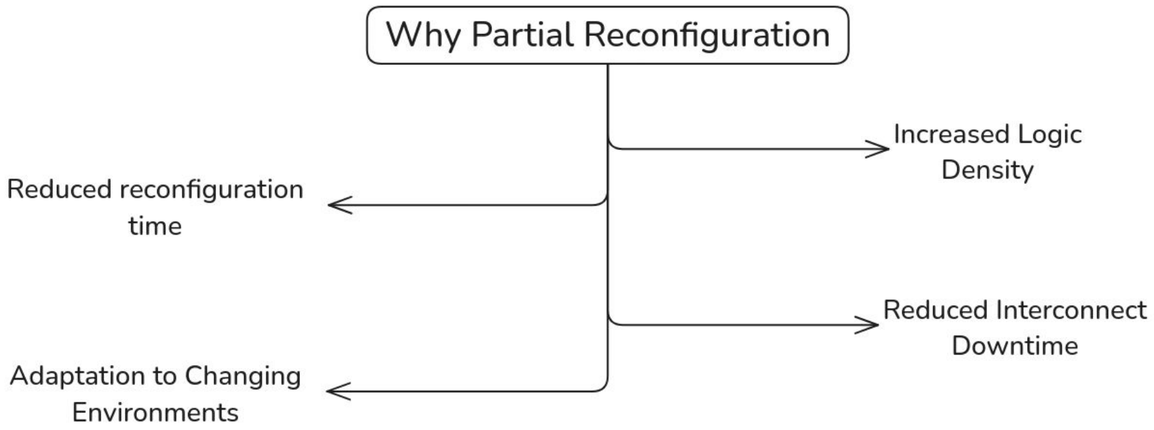
Multi-Context FPGA

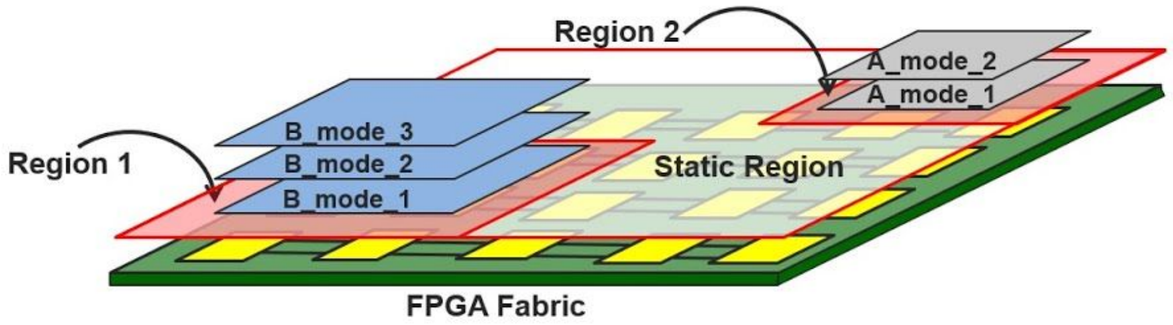
- Multi-Context FPGA - FPGA architecture that incorporates multiple configuration memory layers
- This allows it to store and switch between multiple configuration on-chip
- Addresses the slow reconfiguration time bottle neck by enabling Fast context Switching
- Drawbacks : Consumes more power compared to traditional FPGAs due to multiple config layers



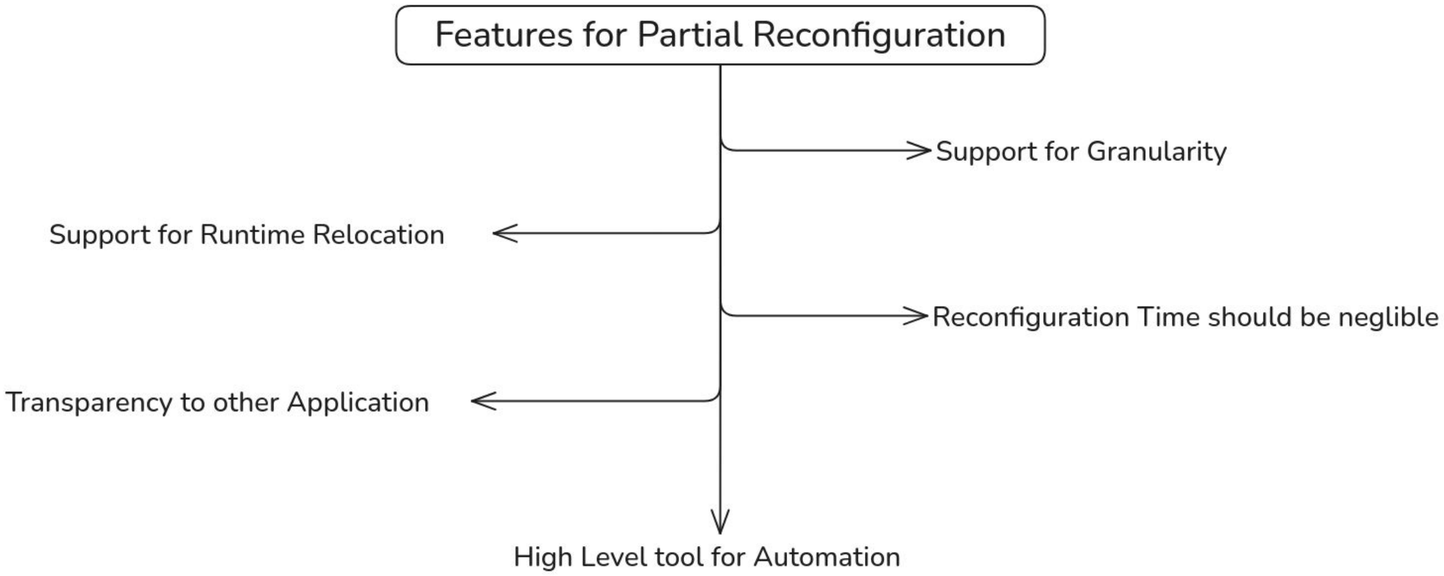
Partial Reconfiguration

- Partial Reconfiguration refers to the modification of one or more regions of the FPGA while the remaining regions are not altered
- Dynamic PR is when this process occurs during runtime of the device with no downtime





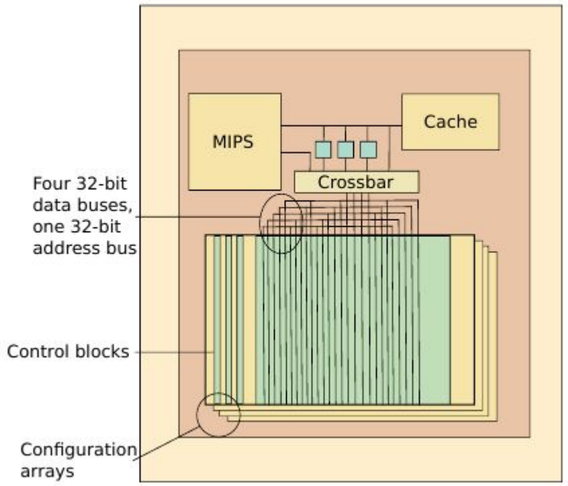
Features of PR



Architecture

GARP Architecture

- GARP consists of a MIPS Processor which controls the reconfiguration process
- On-Chip Reconfigurable hardware called Garp Reconfigurable Hardware which is a co-processing unit
- The Reconfigurable hardware contains 4 Config memories
- The wide data bus between MIPS and the array helps in faster reconfiguration
- Steps in GARP:
    i. Load Configuration
    ii. Copy initial data from register to Array
    iii. Start array execution
    iv. At completion copy data to live registers



a. GARP Architecture

Frames & Tiles

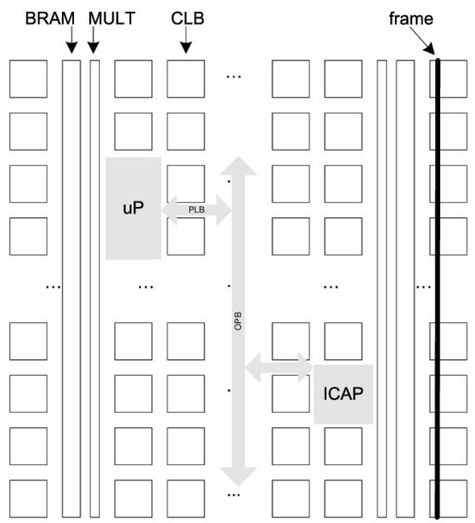
Frames

- Frames are the smallest unit of configuration that can be read or written (fig a)
- They configure a narrow single bit slice of multiple hardware resources
- Initially, each frame's height was same as the device height and 1-bit wide
- Since frames configure a narrow slice of resources, they enable efficient partial reconfiguration.

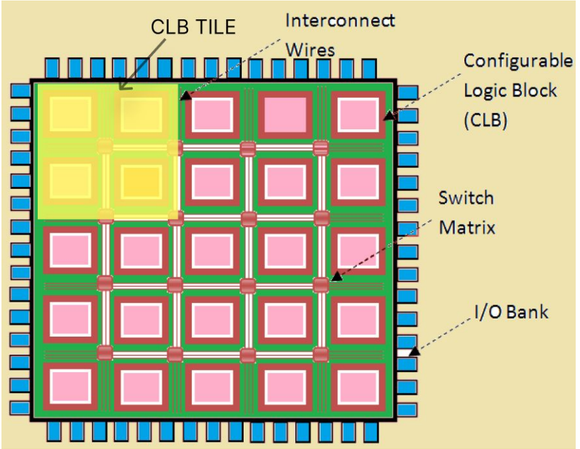
Tiles

- To minimize the need for full-height reconfiguration of the FPGA, the die is divided into multiple tiles, allowing for more localized and efficient partial reconfiguration
- A single CLB tile will contain a certain amount of CLBs similarly for BRAM, DSPs etc.(fig b)

	CLB Tile	DSP Tile	BRAM Tile
Virtex – 5	20	8	4
Virtex – 6	40	8	8
7 Series	50	10	10



a. Virtex II

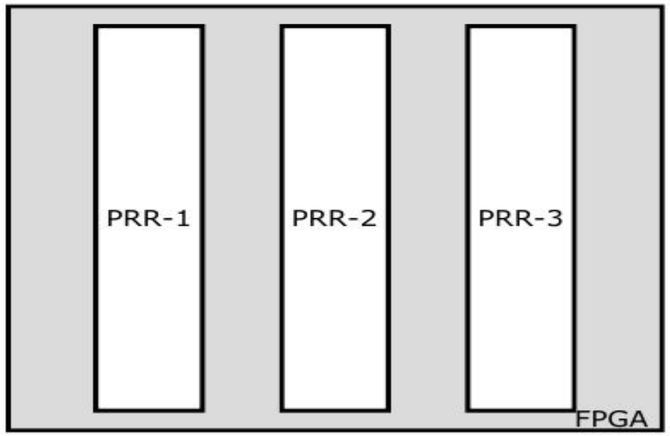


b. CLB Tiles

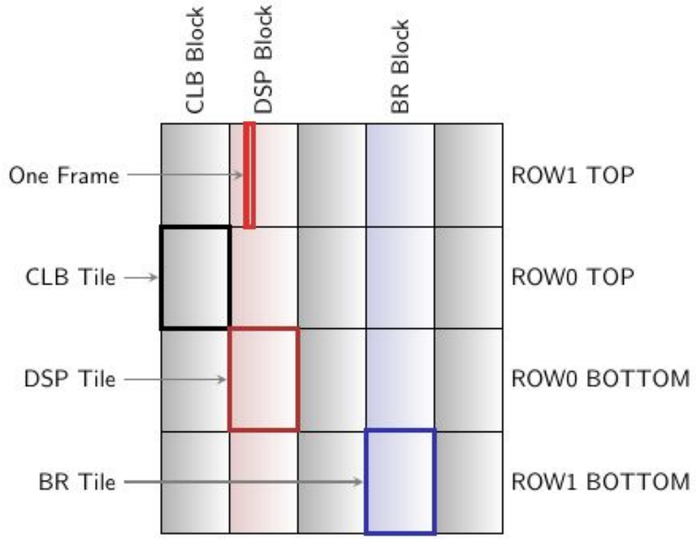
Architecture

Xilinx Virtex PR Architecture

- Frames extend vertically throughout the height of device and are 4 CLBs wide
- Uses Slot Machine Architecture fig(a)
- All PRR should have similar interface with Static region
- Uses buffers as Anchor Logic to avoid glitches during reconfiguration
- Limitation - Number of buffers and their fixed location
- Virtex 4 overcomes this issue by using LUT based multiplexer instead of buffers



a. Slot Machine Architecture

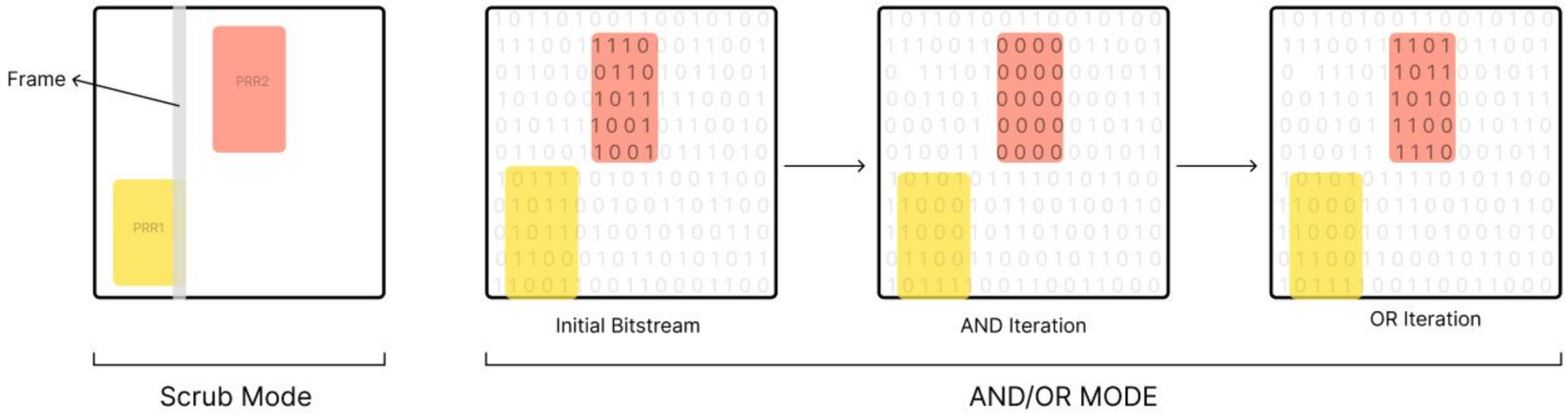


b. Virtex Architecture

Architecture

Altera PR Architecture

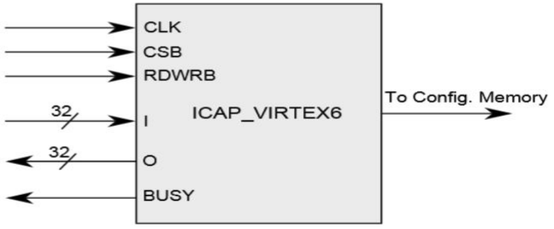
- Two different implementations SCRUB mode and AND/OR method
- SCRUB mode:
  i. Re-Writing LABs(Logic Array Blocks) irrespective of previous frame. The static regions are also scrubbed back.
- AND/OR reconfiguration:
  i. First Iteration, bits corresponding to PRR in frames are set to 0 using AND
  ii. In Second pass for each frame new data is ORed with current value of 0 in PRR and static region bits are ORed with.



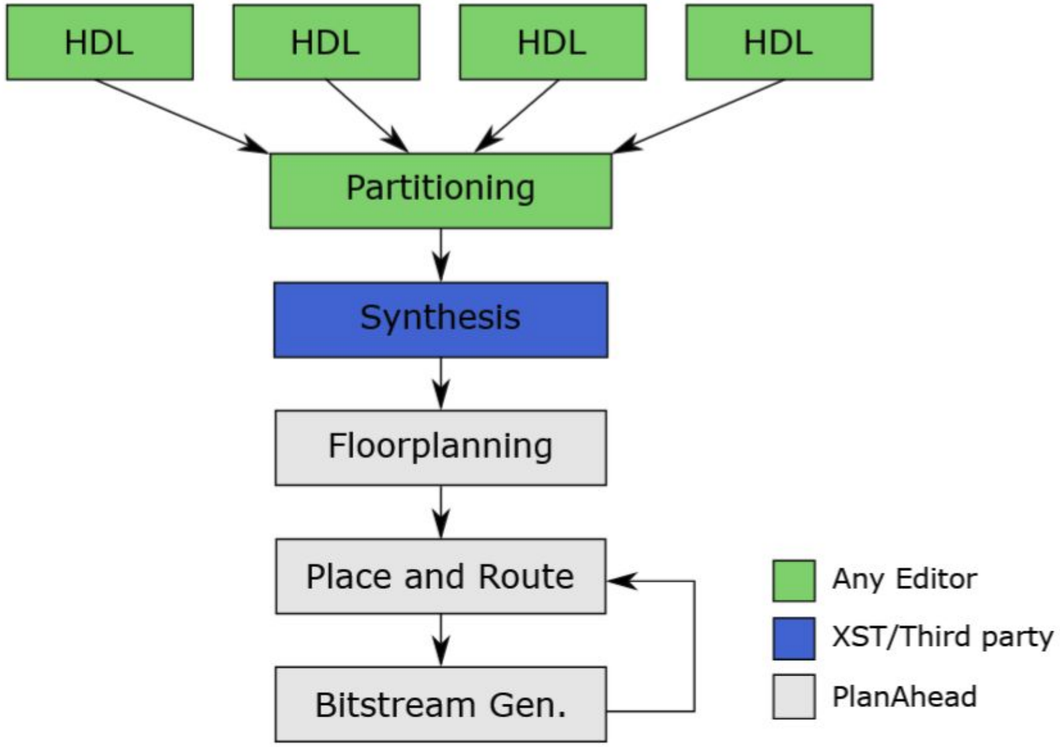
Design, Implementation & Simulation Tool chain

Xilinx

- **PlanAhead:** Manual PRR definition, UCF-based, uses bus macros, supports ISE
- **Vivado:** Pblock-based PRR definition, uses interconnect tiles (no bus macros), supports partial bitstreams
- **Common Aspects:** Manual floorplanning, automated implementation, ICAP/PCAP for reconfiguration
- **Limitations:** No dynamic relocation, static region re-implementation needed on minor changes

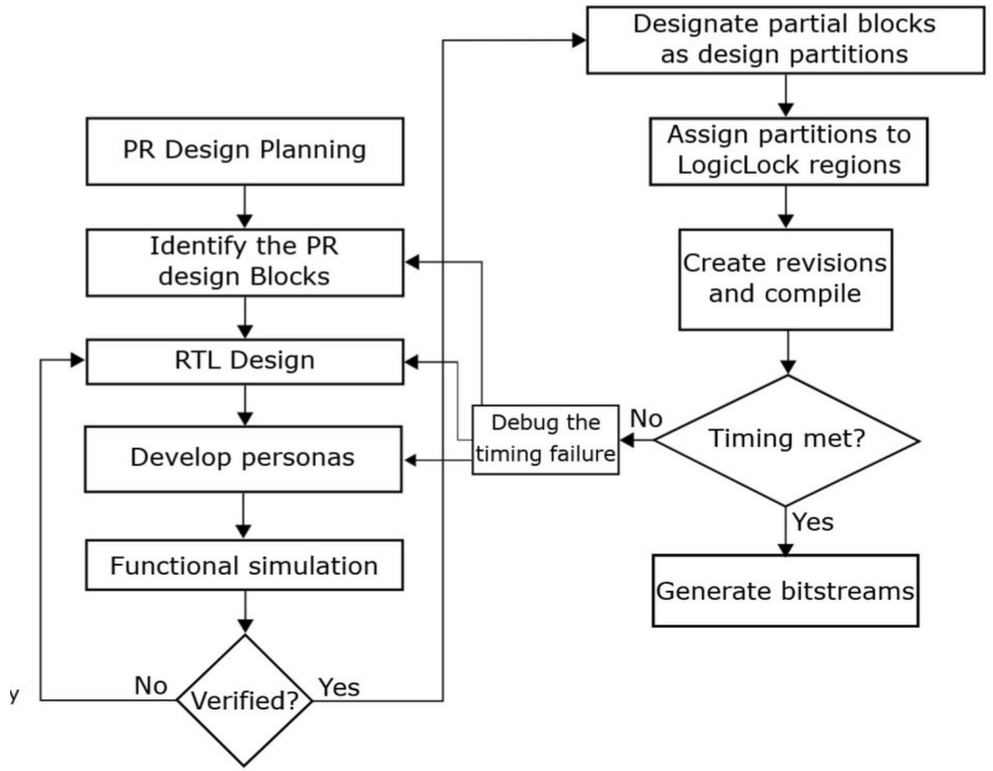


(c)

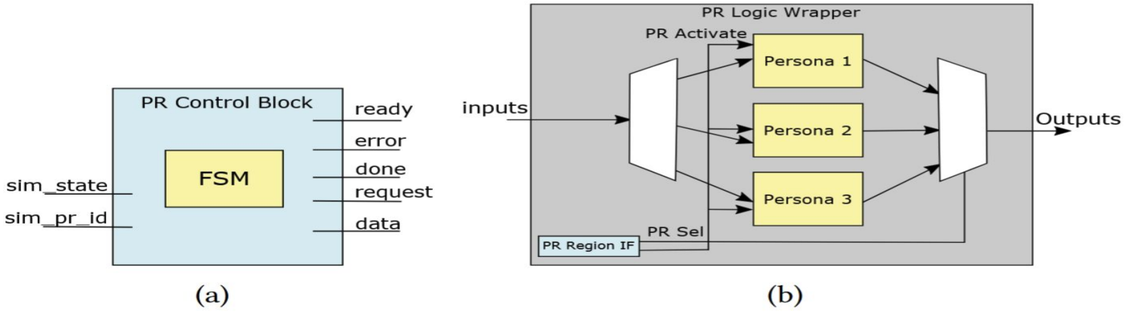


Design, Implementation & Simulation Tool chain

Altera



- **Quartus Flow:** Uses LogicLock regions, incremental compilation, vertical PRR constraints
- **PR Constraints:** PRRs must span full height when partial width; RAM pre-write rules
- **Runtime:** Uses CRC detection, freeze logic, and configuration safety features
- **Multiplexing Method:** Uses **spatial multiplexing**, where multiple PR modules (personas) are placed side-by-side and selected at runtime instead of loading bitstreams.
- **OpenCL SDK:** High-level abstraction, single PRR usage, PCIe-based host control

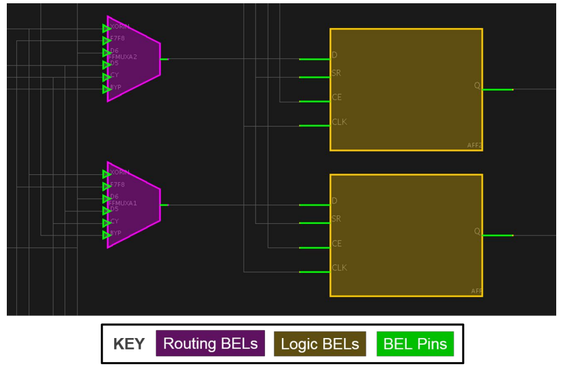


Design, Implementation & Simulation Tool chain

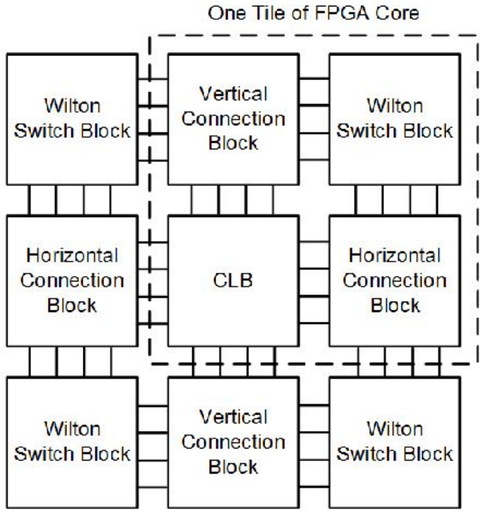
Xilinx vs Altera

Feature	Xilinx	Altera
PRR Flexibility	Rectangular,Pblock-defined	Full-height if partial-width (vertical slicing)
Clock Handling	Interconnect tile-based	LUT-based anchors
Runtime Control	ICAP/PCAP	CRC + Freeze Logic, Mux Method
Simulation Support	DCS, ReSim	Spatial multiplexing, CRC models
Timing-aware sim	Limited (ICAP sim only)	No detailed PR timing

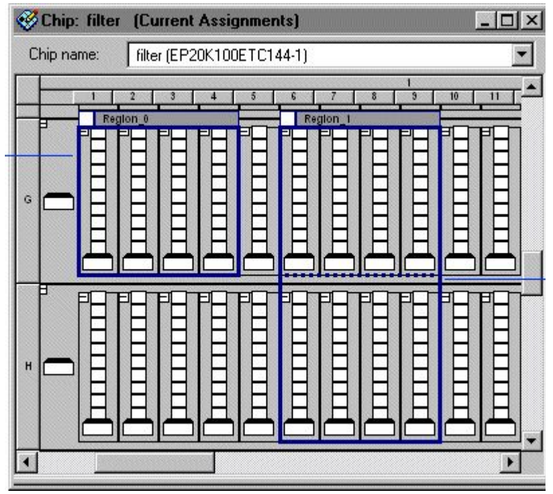
Basic Element of Logic



Interconnect Tiles



LogicLock



Design, Implementation & Simulation Tool chain

Vendor Design and Implementation Tools

Tool	High Lvl Spec. & Partitioning	PRR Definition & Floorplanning	Physical Implementation (PnR)	Circuit Relocation	Runtime Management	Shortcomings
Xilinx PlanAhead	Not supported, Manual (Requires Constraints)	Manual PRR definition, manual floorplanning	Automatic PnR	Not supported	Not supported	Outdated tool, limited automation, Discontinued.
Xilinx Vivado	Not supported, Manual (Requires Constraints)	Manual PRR definition, manual floorplanning	Automatic PnR	Not supported	Not supported	Requires Low-lvl-FPGA architecture knowledge, Complex set-up
Altera Quartus	Not supported, Manual (Requires Constraints)	Manual PRR definition, manual floorplanning	Automatic PnR	Not supported	Not supported	Limited PR granularity, lacks flexibility
Xilinx SDAccel	Supported, Automatic (Via HLS)	Automatic PRR definition, Automatic floorplanning	Automatic PnR	Not supported	Not supported	Limited PnR Customisations
Altera OpenCL	Supported, Automatic (Via HLS)	Automatic PRR definition, Automatic floorplanning	Automatic PnR	Supported	Fully Supported	Lacks low-level PR control

Design, Implementation & Simulation Tool chain

Academic Tools

- **GoAhead**: GUI floorplanner, blocker macros, deep integration with legacy Xilinx flow
- **CoPR**: Zynq-only, kernel tessellation, high-level spec, automatic C-based PR controller
- **OpenPR/PRFlow**: XML/Tcl-based automation, partial integration with vendor flows
- **DyNaP, GePaRD, OSSS+R**: Targeted at specific architectures, explore PR timing & runtime optimization

Design, Implementation & Simulation Tool chain

Academic Design & Implementation Tools

Tool	High Lvl Spec. & Partitioning	PRR Definition & Floorplanning	Physical Implementation (PnR)	Circuit Relocation	Runtime Management	Shortcomings
OpenPR	Not supported, Manual	Manual PRR definition, manual floorplanning	Automatic PnR	Yes	Not supported	No runtime management, manual setup
CoPR	Supported,Semi-Automatic	Semi-Automatic PRR definition, Semi-Automatic floorplanning	Automatic PnR	Yes	Not supported	only supports Zynq SoCs from Xilnx
GoAhead	Not supported, Manual	Manual PRR definition, manual floorplanning	Automatic PnR	Yes	Not supported	No High Lvl Abstraction
Caronte	Supported,Semi-Automatic	Semi-Automatic PRR definition, Semi-Automatic floorplanning	Automatic PnR	Yes	Yes	Lacks Robust runtime Support
GePaRD	Supported, Manual	Manual PRR definition, manual floorplanning	No info	No info	No infor	High complexity, manual PRR setup
PaRAT	Supported, Manual	Manual PRR definition, manual floorplanning	No info	No info	No infor	Requires detailed PRR definitions
OSSS+R	Not supported, Manual	Manual PRR definition, manual floorplanning	No info	No info	No infor	No vendor integration, outdated

Design, Implementation & Simulation Tool chain

Technical Challenges, Toolchain Gaps

Technical Challenges

- **Manual PRR Definition**: Most tools require manual PR region setup.
- **Limited Runtime PR Support**: Only a few tools handle dynamic reconfiguration well.
- **Toolchain Fragmentation**: Different tools specialize in different aspects, leading to inefficiencies.
- **Architecture-Specific:** Most tools are targeted for vendor products.
- Lack of unified research efforts.

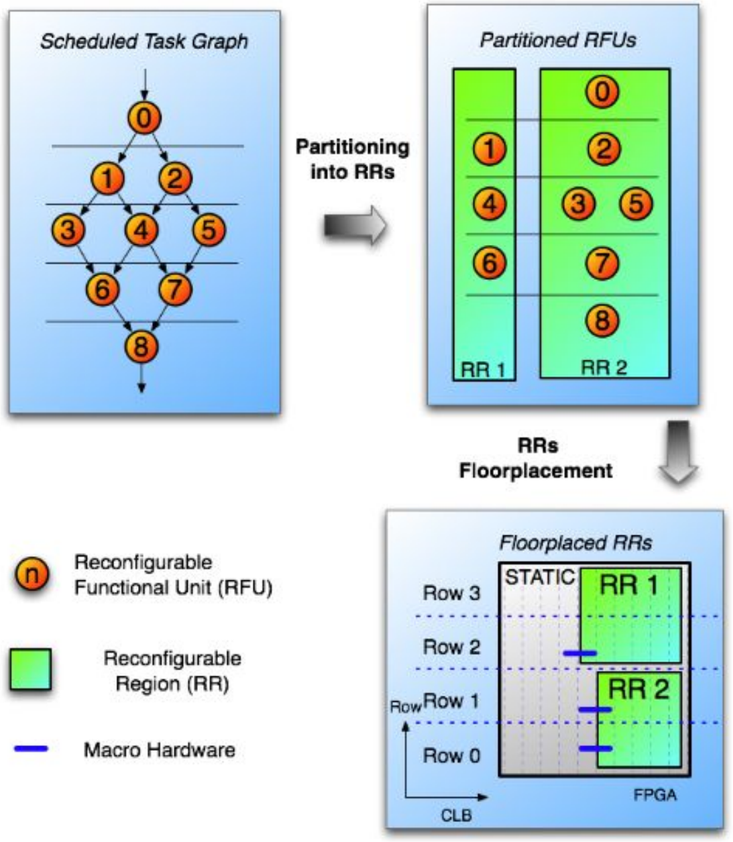
Toolchain Gaps

- **Lack of Standardization**: No unified PR workflow across vendors.
- **Vendor Lock-in**: PR methodologies are highly tool-dependent.
- **High Expertise Requirement**: Most tools demand in-depth FPGA knowledge.

Overhead Reduction Methods

Partitioning

- Mutually exclusive modules are often assigned to the same PRR to optimize resource usage
- Combining modules to the same PRR results in increase in area and reconfiguration time
- An technique to optimise this is to create a graph of dependant tasks for scheduling them and assign the hardware via simulated annealing

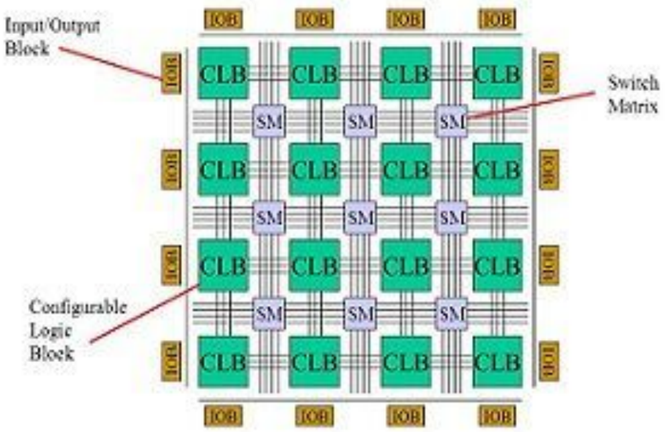


A proposed tool flow that uses graph partitioning

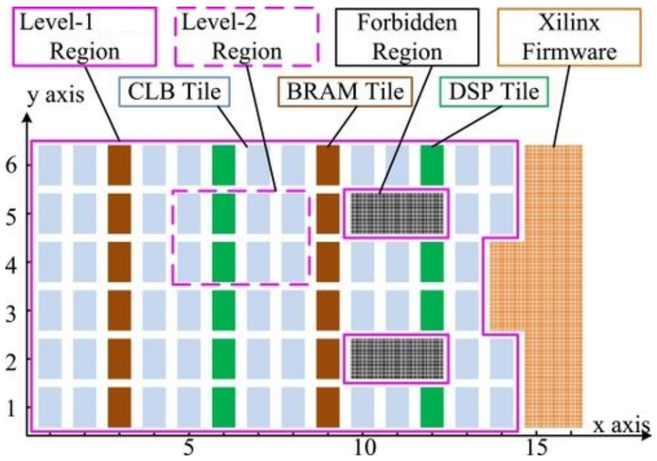
Overhead Reduction Methods

Floorplanning

- Vendor tools lack automatic floorplanning support which means it need to be done manual which can be cumbersome and can lead to suboptimal designs
- Research has gone into automating detailed aspects of floorplanning as well as improving support for heterogeneous FPGAs:
  - Some of the developed tools created partitions with irregular shapes, which were not supported by the fpgas
  - Tools were later developed to generate rectangular shapes as well as adapt to different resource availability in the FPGA
  - Novel approaches like Mixed-Integer Linear Programming was also integrated into the tools. But this approach took significantly higher toolflow time



Homogeneous FPGAs



Heterogeneous FPGAs

Overhead Reduction Methods

Runtime Placement and Configuration File Management

- An approach to reduce toolchain runtime was to enable runtime relocation via placement, routing and bitstream manipulation.
- But the limited processing power of the embedded systems typically used for these online manipulations and the heterogeneous architecture of modern FPGAs became a bottleneck
- The major issue of online PnR tools is lack of portability. Due to architectural variations (even within same family) the tools must be modified for each device

Configuration File Compression

- Reducing the size of the fetched bitstream to improve reconfiguration throughput. This can be done through identifying configuration similarities within and between bitstreams

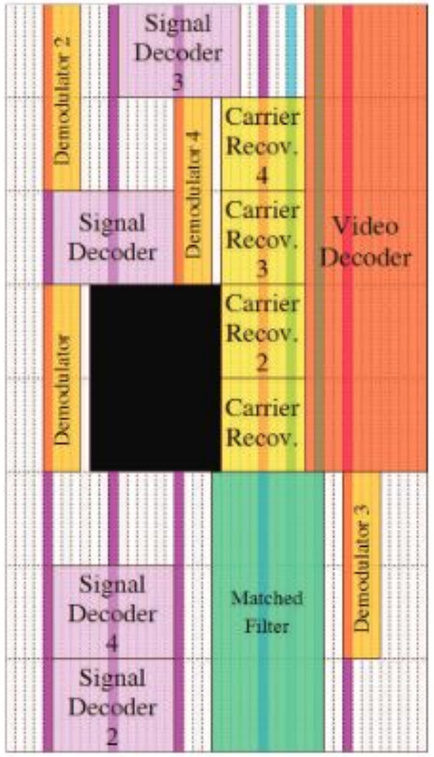


Fig. 5: SDR3 floorplan (9 free-compatible areas)

Runtime Placement Example

Overhead Reduction Methods

High Speed Reconfiguration Controllers

- A reconfiguration controller is responsible of delivering bitstreams to ICAP macro. Vendor provided controllers tend to have much lesser throughput compares to ICAP macro (~10 MBps vs 400 MBps)
- Research into improving the throughput involves techniques like
  - FSL - simplex communication protocol with higher throughput
  - DMAs - Can be used to transfer bitstreams at a much higher throughput
  - Overclocking of ICAP primitive - this technique is device specific
  - PCIe direct streaming - for fetching the bitstreams and reconfiguring through the MCAP

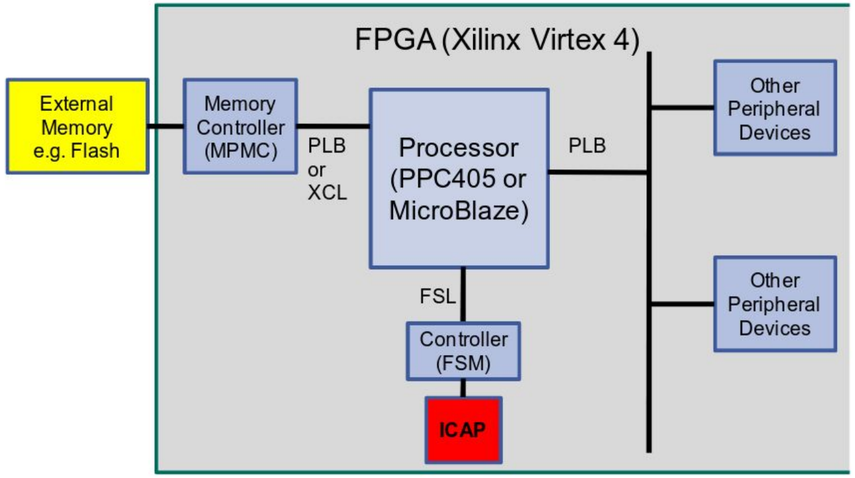


Figure 5. Novel configuration data path through direct FSL connection:

Runtime Management

Reconfigurable Tasks

- Early research assumed FPGAs were composed of homogenous units allowing seamless creation of hardware tasks.
- This was because relocation of modules was an important factor in management of hardware tasks. This was relatively simpler in homogeneous FPGAs
- Heterogeneous nature of modern FPGAs makes this harder to achieve. Some research solutions involved:
  - Maintaining a database of partial bitstreams and all possible PRRs
  - Standardization of PRRs with respect to each other so that the address in the configuration file can be modified to handle hardware tasks
  - Configuration caching also helps in speed up of reconfiguration times in fixed sequence of reconfigurations

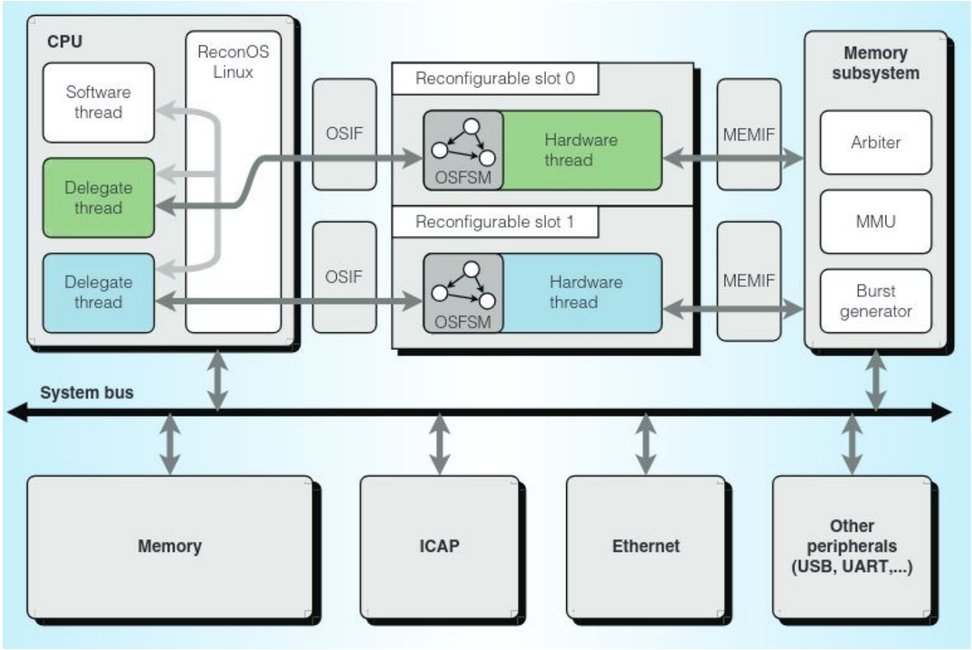
Runtime Management

Software Based Management

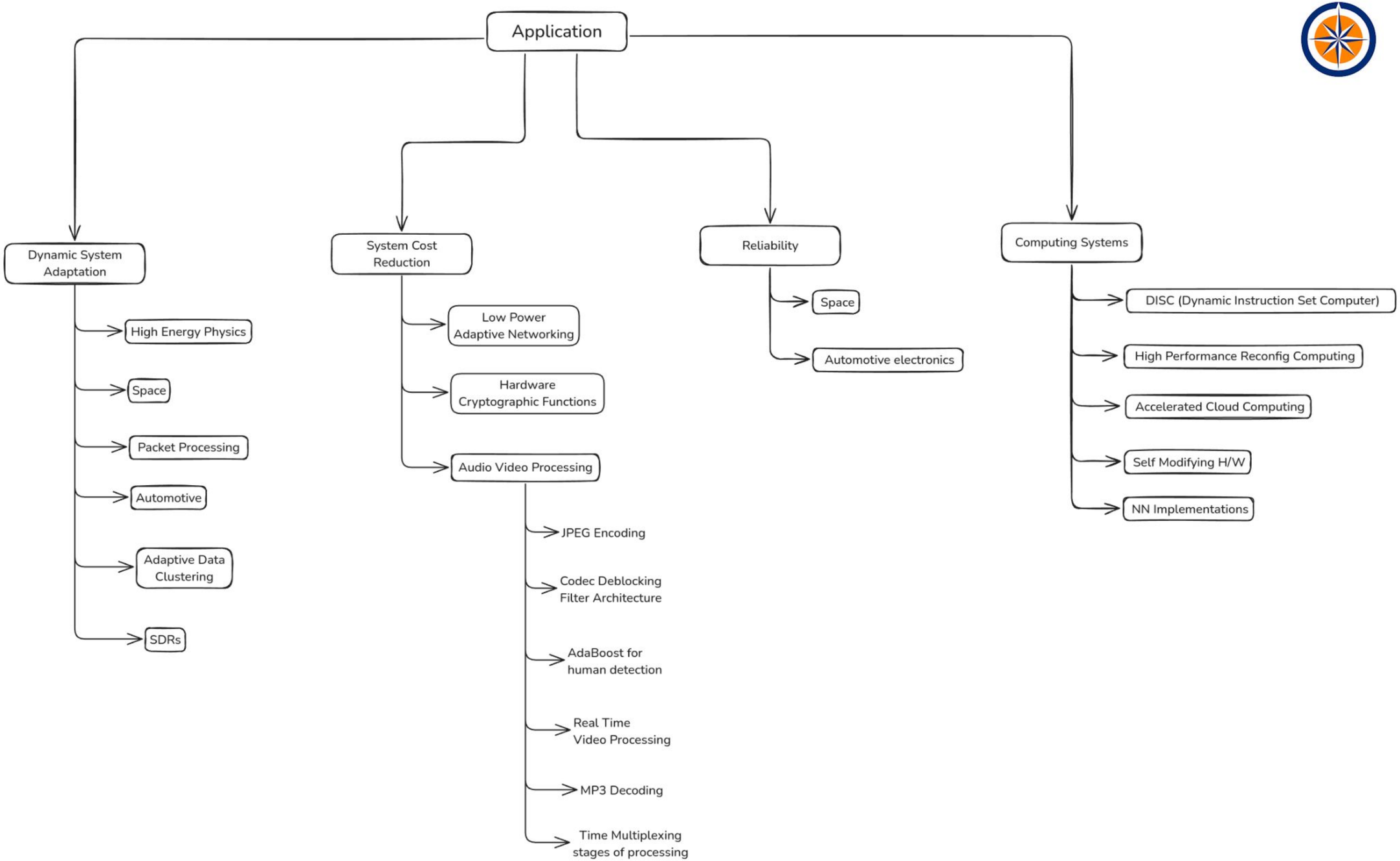
- Abstraction of reconfiguration from the programmer by using API calls similar to OpenCL frameworks
- The drawback of such an approach is the overhead introduced through multiple software layers
- Some implementations can be found in:
  - GNU/Linux
  - ReconOS

Custom Hardware Based Management

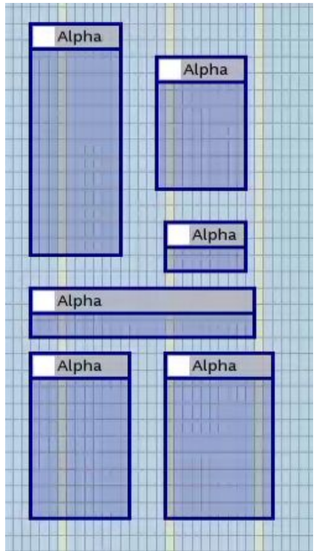
- High reconfiguration throughput via controllers with access to DMA
- This comes at the cost of flexibility that can be found in the software approach
- Optimal option would be use higher level (abstraction) software combined with lower level hardware management blocks



ReconOS Architecture


Application
Dynamic System Adaptation
High Energy Physics
Space
Packet Processing
Automotive
Adaptive Data Clustering
SDRs
System Cost Reduction
Low Power Adaptive Networking
Hardware Cryptographic Functions
Audio Video Processing
JPEG Encoding
Codec Deblocking Filter Architecture
AdaBoost for human detection
Real Time Video Processing
MP3 Decoding
Time Multiplexing stages of processing
Reliability
Space
Automotive electronics
Computing Systems
DISC (Dynamic Instruction Set Computer)
High Performance Reconfig Computing
Accelerated Cloud Computing
Self Modifying H/W
NN Implementations

Future Directions

- Emerging interest in use of PR in data centers
- Adaptation of OpenCL for use with FPGAs
- Uses in autonomous systems like automotives, aerospace, communication
- Application of AI in automating PR definitions
- Unified tool flow for PR

Thank You